in their emails. In the case of a plain-text email, we want to strip out the rich text from our response. Action Text provides a method to do that, called `to_plain_text`, which we can use in `app/views/support_request_mailer/respond.text.erb`, like so:

rails7/depot_tc/app/views/support_request_mailer/respond.text.erb

```
<%= @support_request.response.to_plain_text %>

---

<%= @support_request.body %>
```

The last step is to add a call to our mailer when we update the `SupportRequest`:

```
class SupportRequestsController < ApplicationController
  def index
    @support_requests = SupportRequest.all
  end

  def update
    support_request = SupportRequest.find(params[:id])
    support_request.update(response: params.require(:support_request)[:response])
»   SupportRequestMailer.respond(support_request).deliver_now
    redirect_to support_requests_path
  end
end
```

Now, if you restart your server and respond to a support request, you'll see the mail printed out in your log, and you should see that the plain-text part of the email is free of formatting.

What We Just Did

- We configured and set up Action Mailbox to allow our app to receive support emails. We saw how to configure Rails to inspect each incoming email and route it to the right bit of code, called a mailbox.

- We also configured Active Storage, which Rails uses to store the raw emails it processes. With it set up, we could easily access cloud storage for any other purpose we might need.